

Classes and Object-Oriented Programming

Fundamental Concepts of OOP



Object-oriented programming brings several useful concepts like **inheritance, encapsulation, abstraction, and polymorphism, modularity** .

Encapsulation



Encapsulation is a process of combining data members and functions in a single unit called class.

This is to prevent the access to the data directly, the access to them is provided through the functions of the class.

It is one of the popular feature of Object Oriented Programming(OOPs) that helps in data hiding

Abstraction



Through the process of **abstraction**, a programmer hides all but the relevant data about an object in order to reduce complexity and increase efficiency.

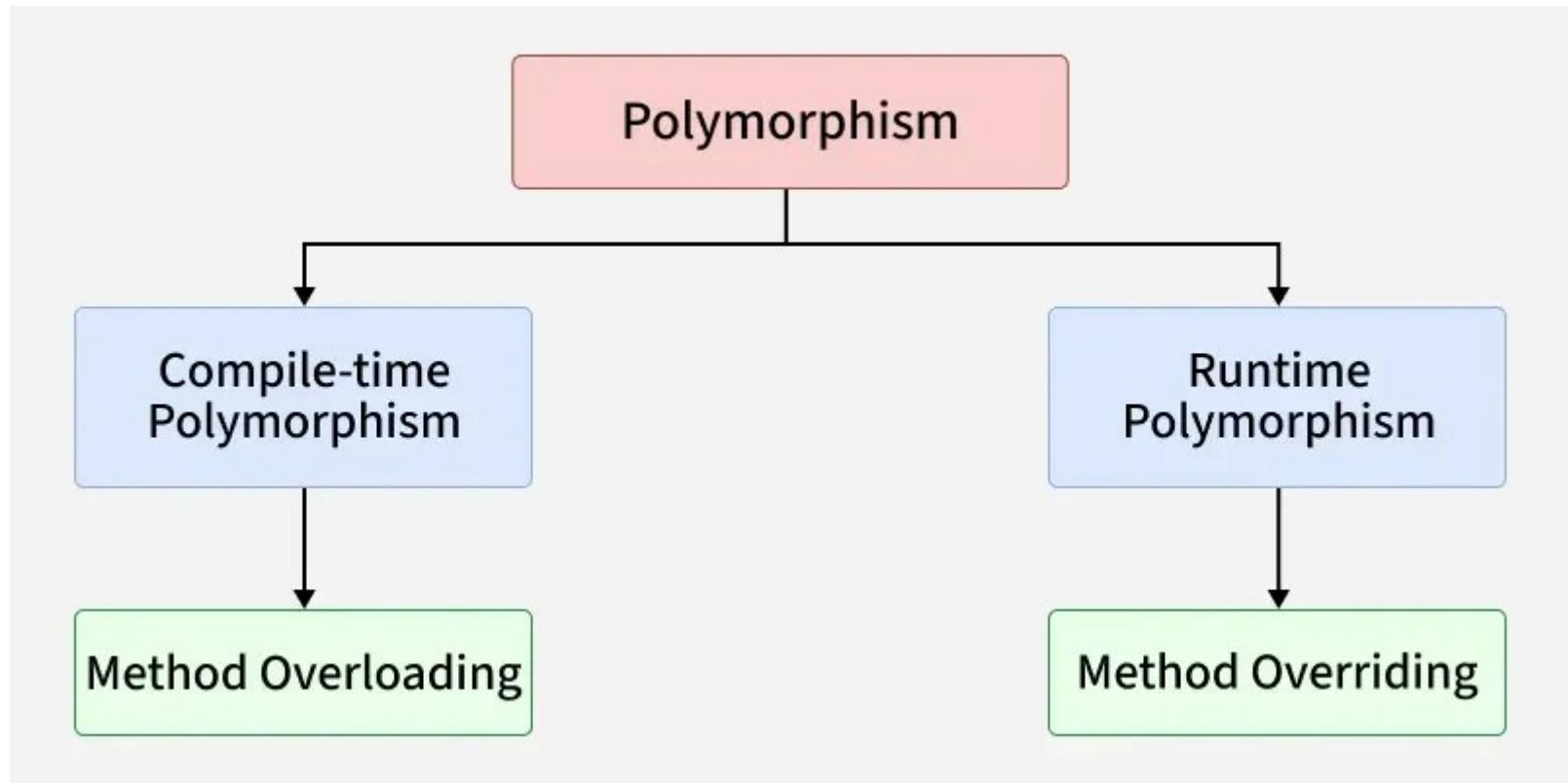
Polymorphism

Polymorphism means "**many forms**". It refers to the ability of an entity (like a function or object) to perform different actions based on the context.

Example of Polymorphism

A remote control can operate multiple devices like a TV, AC or music system. You press the power button and each device responds differently TV turns on, AC starts cooling, music system plays music.

Polymorphism



Inheritance



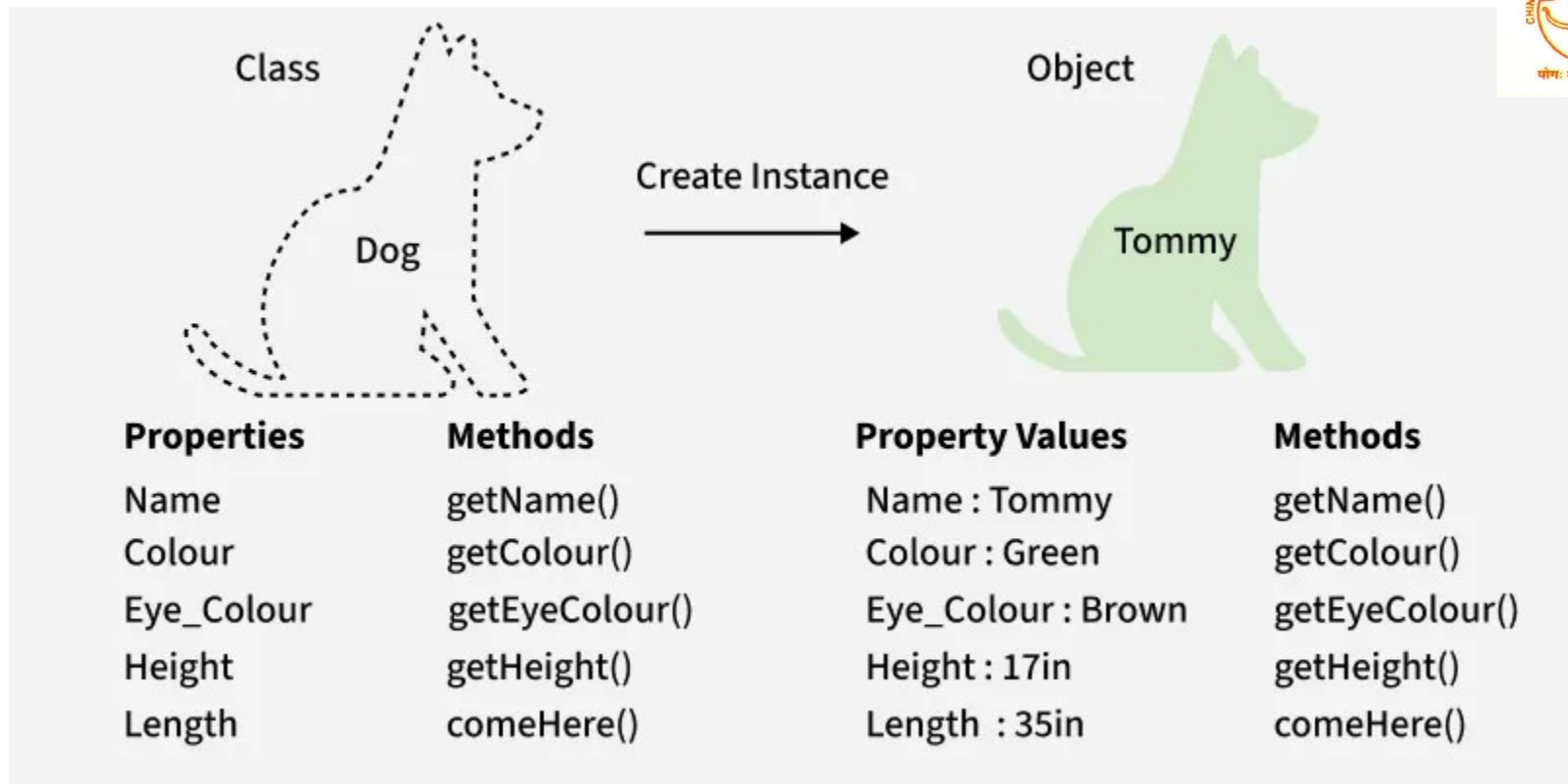
The capability of a **class to derive properties and characteristics from another class is called Inheritance.**

Inheritance is one of the most important feature of Object Oriented Programming.

Classes



- Classes are the mechanism used to create new kinds of objects
- Class creates a user-defined data structure, which holds its own data members and member functions, which can be accessed and **used by creating an instance of that class.**



Classes and Objects

(Here Dog is the Base Class and Bobby is Object)

Classes



OOP allows representation of real-life objects as software objects

Object: A single software unit that combines attributes and methods

Attribute: A "characteristic" of an object; like a variable associated with a kind of object

Method: A "behavior" of an object; like a function associated with a kind of object

Class: Code that defines the **attributes** and **methods** of a kind of object (A class is a collection of variables and functions working with these variables)

Classes



- A class defines a set of *attributes* that are associated with, and shared by, a collection of objects known as *instances*
- A class is most commonly a collection of *functions* (known as methods), *variables* (which are known as class variables)
- A class is defined using the **class** statement.
- The body of a class contains a series of statements that execute during class definition

```
class Dog:  
    sound = "bark"
```

```
# Creating object  
dog1 = Dog()  
print(dog1.sound)
```

Constructor



Constructor: A special method that is automatically invoked when a new object is created

- Usually write one in each class
- **Usually sets up the initial attribute values of new object in constructor**

- `__init__(self)`

Is special method name indicate the constructor of a class

```
class A:  
    def __init__(self):  
        print("Constructor is called..")
```

Automatically executed by `ob=A()`

Constructor



Two types of Constructors are available:

- Default constructor – This constructor doesn't accept any argument
- Parameterized constructor- constructor with parameter is known as parameterized constructor.

Classes



```
class Student:
    def __init__(self, name):
        self.name = name
    def talk(self):
        print ("Hi, I'm", self.name)

std1 = Student("Hari")
std1.talk()
std2 = Student("Arun")
std2.talk()
```

Class: Code that defines the **attributes** and **methods** of a kind of object

Instantiate: To create an object; A single object is called an **Instance**

Constructor



Initializing Attributes

```
class Student:
    def __init__(self, name):
        self.name = name
...
std1 = Student("Hari")
```

We Can have object's attributes **automatically** created and initialized through constructor.

`self` – first parameter in every instance method

- » `self` receives reference to new Student object
- » `name` receives "Hari"
- » `self.name = name` creates the attribute name for object and sets to "Hari"
- » `std1` gets new Student object

Accessing Attributes

```
class Student:  
    ...  
    def talk(self):  
        print("Hi, I'm", self.name)  
    ...  
std1.talk()  
print(std1.name)
```

- Assessing attributes using methods: **talk()**
 - » Uses a Student object's name attribute
 - » Receives reference to the object itself into `self`
- Accessing Attributes Directly

Destructor



Destructors are called when an object gets destroyed.

In Python, destructors are not needed as much needed in C++ because Python has a garbage collector that handles memory management automatically.

- **`__del__(self)`**

Is special method name indicate the destructor of a class

Constructor and Destructor

```
class Employee:
```

```
    # Initializing
```

```
    def __init__(self):  
        print('Employee created.')
```

```
    # Deleting (Calling destructor)
```

```
    def __del__(self):  
        print('Destructor called, Employee deleted.')
```

```
obj = Employee()
```

```
del obj
```

- The `__init__()` function is called when create a new object.
- `__del__()` when after the program ended or when all the references to object are deleted

Creating array of Class Objects



- We can create list of object in Python by appending class instances to list.
- By this, every index in the list can point to instance attributes and methods of the class and can access them

```
std=[]  
for i in range(0,3):           #it creates 3 objects  
    std.append(Student())  
  
for s in std:  
    s.read()
```

Class Attribute vs. Instance Attribute

- Python provides two scopes for attributes: class attributes and instance attributes.

An **instance attribute** is a Python variable belonging to one, and only one, object. This variable is only accessible in the scope of this object and it is defined inside the constructor function, `__init__(self,..)` of the class

A **class attribute** is a Python variable that belongs to a class rather than a particular object.

It is shared between all the objects of this class and it is defined outside the constructor function, `__init__(self,...)`, of the class.

Class Attribute vs. Instance Attribute

```
class Person:
```

```
    no_of_person=0
```

```
//Class Attribute
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
//Instance attribute
```

```
        self.age = age
```

```
        Person.no_of_person +=1
```

```
p1 = Person("John", 36)
```

```
p2= Person("Alex",40)
```

```
print(p1.name)
```

```
print(p1.age)
```

```
print(Person.no_of_person)
```

Note: The `__init__()` function is called automatically every time the class is being used to create a new object.